

Server 1 — filesystem (escopo: código-fonte, specs e skills)

Pasta/escopo: `./src`, `./specs`, `./skills`

O que expõe:

- Leitura e escrita de arquivos TypeScript em `./src` (handlers, services, pipeline, bot, shared)
- Leitura e edição das specs SDD (`requirements.md`, `plan.md`, `tasks.md`) dos 5 módulos em `./specs`
- Leitura e edição das skills do projeto organizadas em `./skills/foundation/`, `./skills/domain/` e `./skills/artifact/`
- Listagem de diretórios para navegação na estrutura do repositório

Quem consome:

- **Copilot / Claude Code** — geração e edição de código de produção, criação de skills, evolução de specs
- **Dev Pleno e Dev Sênior** — geração assistida de tasks, scaffolding de módulos, criação de testes

Por que o escopo é restrito a essas pastas: evita que o agente acesse inadvertidamente `./infra` (Bicep) ou `./prompts` ao executar tarefas de código, respeitando o princípio de menor privilégio.

Server 2 — filesystem (escopo: documentação de negócio)

Pasta/escopo: `./docs/novatech/` (equivalente ao Anexo A — POL-001, PROC-042 v1 e v2, SLA-2024, FAQ-Atendimento)

O que expõe:

- Acesso somente-leitura à documentação normativa da NovaTech: políticas (POL), procedimentos (PROC), tabelas de SLA e FAQ
- Os mesmos 5 documentos que alimentam o pipeline de RAG, disponíveis como arquivos `.md` individuais
- Contexto para o agente entender regras de negócio *antes* de gerar código que as implementa (ex: ao escrever `response-validator.ts`, entender as regras de contradição do PROC-042)

Quem consome:

- **Product Specialist** — consulta ao escrever `requirements.md` de cada módulo
- **Tech Lead** — referência ao escrever ADRs e `plan.md`
- **Agentes de IA** — contexto de negócio para geração de testes com fixtures realistas, criação de `golden-queries.json` e validação de lógica do prompt

Nota crítica: este escopo replica o papel que o Confluence teria em produção. A separação em instância própria do `filesystem` impede que o agente misture documentação de negócio com código ao executar buscas.

Server 3 — filesystem (escopo: corpus de retrieval)

Pasta/escopo: `./data/retrieval-corpus/` (equivalente ao Anexo B — chunks extraídos do pipeline de RAG)

O que expõe:

- Os chunks pré-processados com metadados (`source`, `version`, `vigencia`) que o Azure AI Search retornaria por similaridade
- O mapa de cobertura pergunta → chunks esperados (gabarito de retrieval)
- As armadilhas documentadas (contradição PROC-042/v2, FAQ como fonte frágil, tier inexistente)
- Fixtures prontas para `tests/fixtures/chunks.ts` e para `prompts/eval/golden-queries.json`

Quem consome:

- **QA** — montagem de cenários de teste de retrieval e avaliação de respostas do assistente
- **Agentes de IA** — simulação do comportamento do pipeline durante desenvolvimento (antes de ter Azure AI Search real), geração de testes de integração com dados realistas
- **Tech Lead** — validação do `prompt-builder.ts` e do `response-validator.ts` contra casos reais

Por que separado do escopo de negócio: os chunks são artefatos do pipeline (resultado de processamento), não a documentação fonte. Misturá-los causaria confusão entre "o que a NovaTech diz" e "o que o retrieval retorna", que são camadas diferentes — exatamente a distinção que o Anexo B ressalta.

Server 4 — `git`

Pasta/escopo: `.` (raiz do repositório local `novatech-assistant`)

O que expõe:

- Histórico de commits para rastreabilidade de decisões
- Diffs entre versões de arquivos (útil para comparar evoluções do `system-prompt.md` e dos ADRs)
- Status de branches e arquivos modificados/staged
- Autoria e timestamps de cada commit (importante para o `prompt-changelog.md` e ADRs)

Quem consome:

- **Tech Lead** — revisão de histórico antes de aprovar PRs, comparação de versões de specs
- **Agentes de IA** — contexto de o que já foi implementado antes de sugerir código, evitando retrabalho ou conflito com decisões anteriores registradas em commits
- **Delivery Manager** — visibilidade de progresso sem precisar abrir cada arquivo

Nota: o server de GitHub foi deliberadamente excluído (arquivado upstream e requer token externo). O `git` local cobre 100% das necessidades desta fase, que opera sem remoto.

Server 5 — `memory`

Pasta/escopo: grafo local em memória (sem pasta física; persiste enquanto a sessão do agente estiver ativa)

O que expõe:

- **Linguagem ubíqua do domínio:** entidades canônicas como `Chunk`, `QueryRequest`, `AssistantResponse`, `SourceCitation`, `DocumentTier`, `ContradicaoFlag` — o bounded context definido nesta fase
- **Decisões arquiteturais resumidas:** os 4 ADRs (modelo LLM, estratégia de contexto, documentos contraditórios, build vs buy) como nós persistentes acessíveis sem reler os arquivos
- **Restrições e regras do projeto:** context budget (4K system + 8K chunks + histórico 3 turnos), stack canônica (TypeScript/React/Bicep), convenções de nomenclatura
- **Estado de progresso da sessão:** quais specs já foram escritas, quais skills já existem, quais testes estão pendentes

Quem consome:

- **Agentes de IA** — manutenção de contexto coerente ao longo de uma sessão longa de desenvolvimento, sem precisar re-injetar os 4 ADRs e o cenário a cada prompt
- **Tech Lead** — registro rápido de decisões tomadas durante a sessão que ainda não viraram ADR formal
- **Todos os papéis** — glossário compartilhado que evita o problema que a NovaTech já tem hoje: cada pessoa chamando a mesma coisa de um nome diferente

Server 6 — `everything`

Pasta/escopo: não aponta para pasta — expõe as primitivas do próprio protocolo MCP

O que expõe:

- Exemplos de `tools`, `resources` e `prompts` do protocolo MCP em forma exercitável
- Echo, geração de dados de teste, sampling de primitivas
- Ambiente seguro para o time aprender como agentes invocam ferramentas MCP antes de conectar servers reais

Quem consome:

- **Tech Lead e Devs** — aprendizado do protocolo antes de implementar integrações MCP customizadas (ex: um futuro server para Azure AI Search)
- **Agentes de IA** — validação de que o ambiente MCP está funcionando antes de executar tarefas reais

Visão consolidada

Necessidade do projeto	Server	Escopo/pasta	Consumidores principais
Ler/escrever código, specs e skills	<code>filesystem</code> #1	<code>./src</code> , <code>./specs</code> , <code>./skills</code>	Devs, agentes de IA
Consultar documentação normativa da NovaTech	<code>filesystem</code> #2	<code>./docs/novatech/</code>	Product Specialist, Tech Lead, agentes
Simular retrieval e montar fixtures de teste	<code>filesystem</code> #3	<code>./data/retrieval-corpus/</code>	QA, agentes, Tech Lead

Necessidade do projeto	Server	Escopo/pasta	Consumidores principais
Histórico de commits, diffs, branches	git	. (raiz local)	Tech Lead, Delivery Manager, agentes
Linguagem ubíqua, ADRs resumidos, estado de sessão	memory	grafo local (sem pasta)	Todos os papéis, agentes
Aprendizado do protocolo MCP, testes de primitivas	everything	— (protocolo)	Tech Lead, Devs

Uma observação sobre o filesystem em múltiplas instâncias

O Anexo C mostra uma configuração com um único filesystem apontando para ./src ./specs ./skills ./docs ./data. Para o nível de controle que este projeto exige — especialmente pela presença de documentação com contradições conhecidas e chunks que simulam um sistema externo — vale considerar separar em instâncias nomeadas (filesystem-code, filesystem-docs, filesystem-corpus). O mcp.json suporta múltiplos servers com nomes distintos; o custo é zero e o benefício é que o agente (e o time) sabem exatamente qual "lente" está sendo usada em cada tarefa.